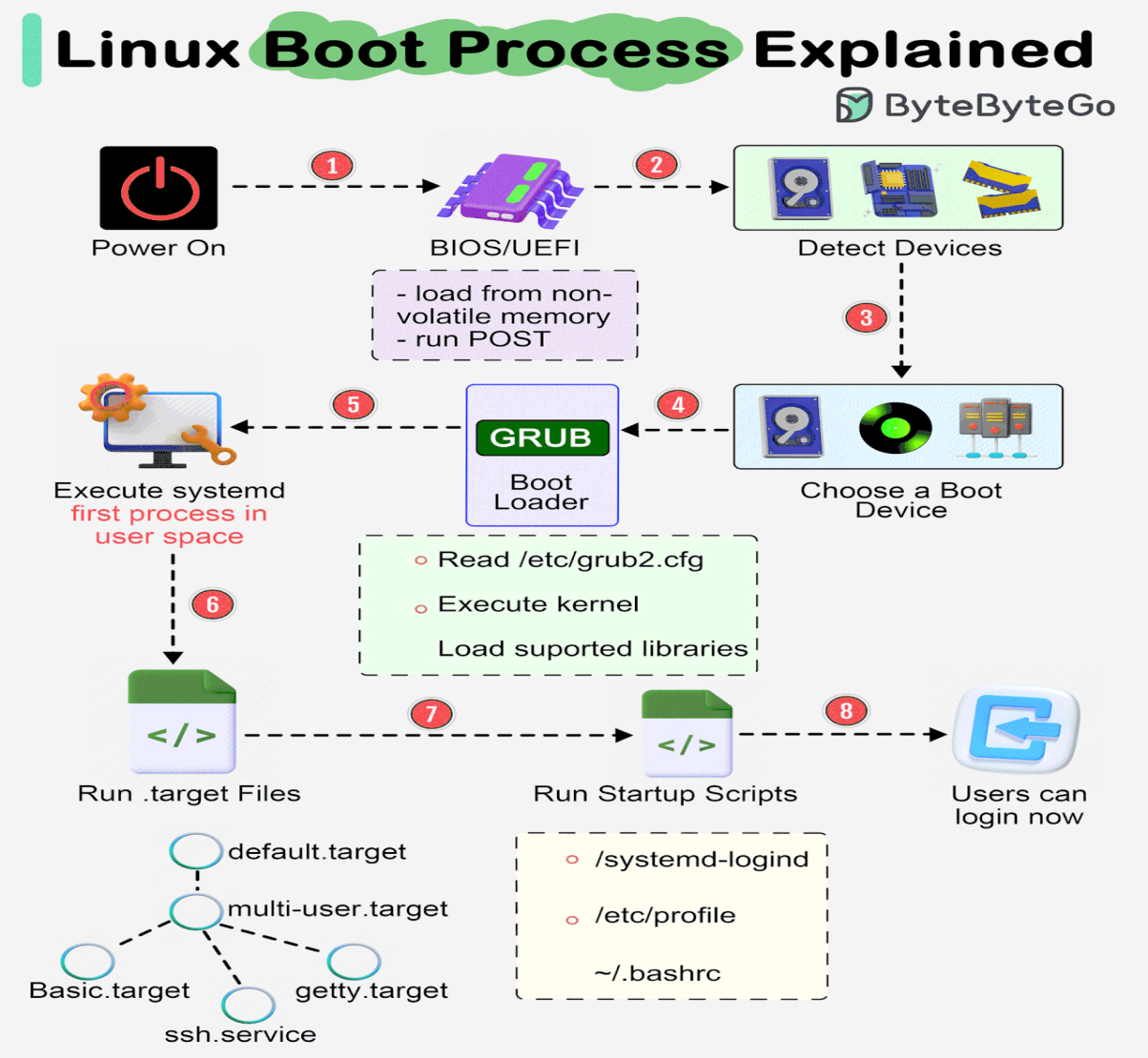
# Le démarrage Linux

« Démarrer Linux, c'est une chaîne d'étapes. Chaque étape prépare la suivante. »

### Schéma logique

```
PC allumé
    ↓
BIOS / UEFI
    ↓
Bootloader (GRUB)
    ↓
Noyau Linux
    ↓
systemd
    ↓
Services + Interface utilisateur
```



# Étape 1 : BIOS / UEFI (le premier déclencheur)

« Le BIOS ou l'UEFI est le **réveil du PC**. Il vérifie le matériel et cherche quoi démarrer. »

### 📌 Rôle

- Vérifie le matériel (RAM, disque, clavier…)
- Cherche un **périphérique de démarrage**
- Lance le **bootloader**

### BIOS vs UEFI (simplifié)

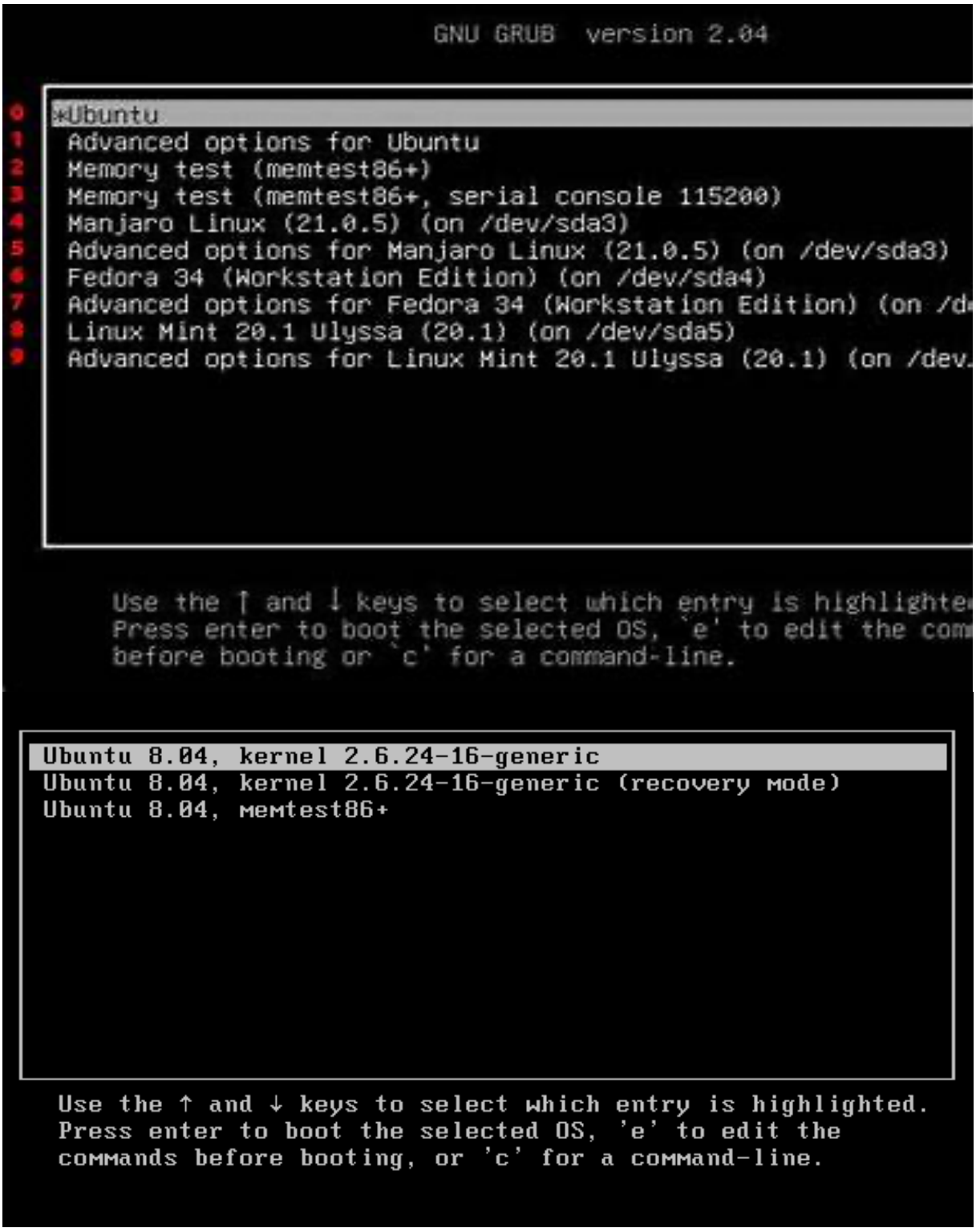
| BIOS | UEFI |
|---|---|
| Ancien | Moderne |
| Disques ≤ 2 To | Disques > 2 To |
| Lent | Rapide |
| Mode texte | Interface graphique |

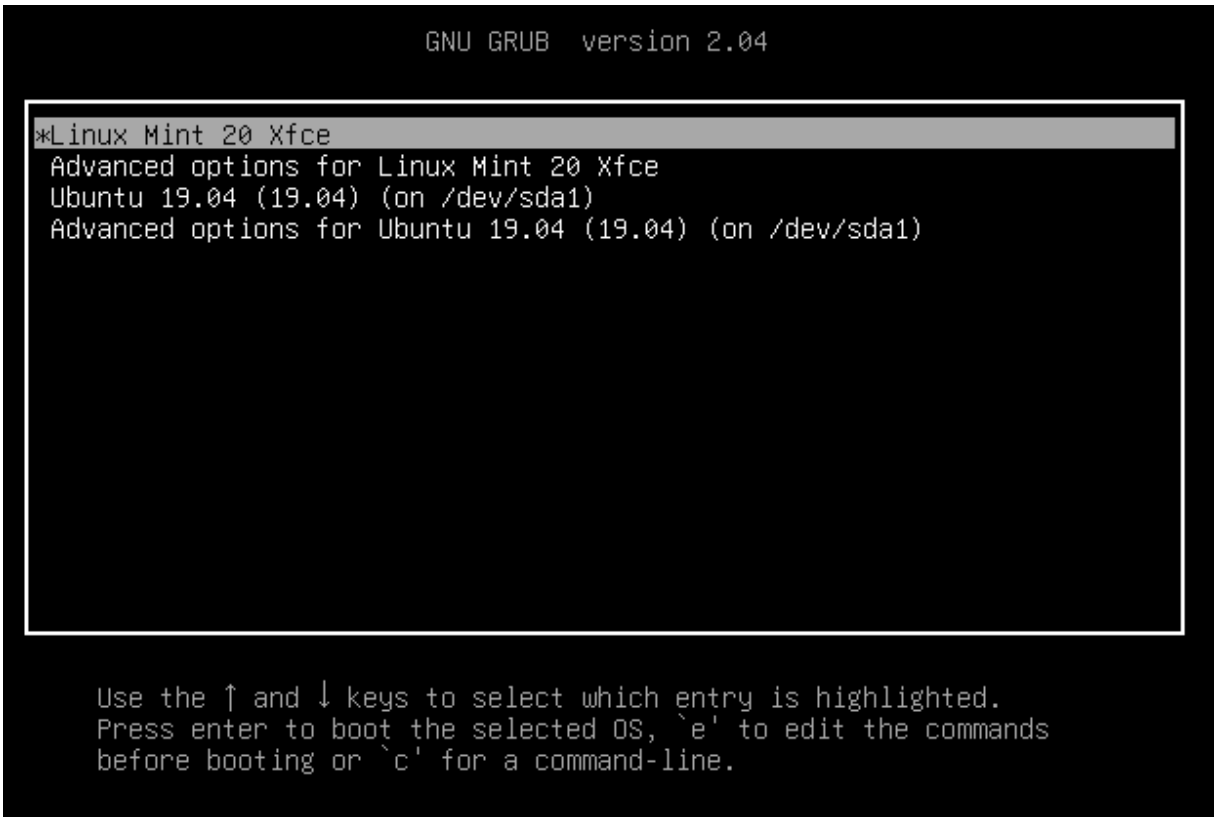
# Étape 2 : Le Bootloader (GRUB)

« GRUB est comme un menu au restaurant : il vous demande **quel système ou quel noyau** vous voulez démarrer. »

### 📌 Rôle de GRUB

- Affiche le menu de démarrage
- Permet de choisir :
  - Linux
  - Windows
  - Ancien noyau
- Charge le **noyau Linux en mémoire**


GNU GRUB version 2.04
*Ubuntu
Advanced options for Ubuntu
Memory test (memtest86+)
Memory test (memtest86+, serial console 115200)
Manjaro Linux (21.0.5) (on /dev/sda3)
Advanced options for Manjaro Linux (21.0.5) (on /dev/sda3)
Fedora 34 (Workstation Edition) (on /dev/sda4)
Advanced options for Fedora 34 (Workstation Edition) (on /d
Linux Mint 20.1 Ulyssa (20.1) (on /dev/sda5)
Advanced options for Linux Mint 20.1 Ulyssa (20.1) (on /dev.
Use the ↑ and ↓ keys to select which entry is highlighte
Press enter to boot the selected OS, `e' to edit the com
before booting or `c' for a command-line.
Ubuntu 8.04, kernel 2.6.24-16-generic
Ubuntu 8.04, kernel 2.6.24-16-generic (recovery mode)
Ubuntu 8.04, memtest86+
Use the ↑ and ↓ keys to select which entry is highlighted.
Press enter to boot the selected OS, 'e' to edit the
commands before booting, or 'c' for a command-line.




---

# Étape 3 : Le noyau Linux (Kernel)

« Le noyau est le chef d'orchestre : il gère le processeur, la mémoire et les périphériques. »

### 📌 Rôle du noyau

- Détecte le matériel
- Monte le système de fichiers racine `/`
- Lance le **processus PID 1**

---

# Étape 4 : systemd (le chef d'orchestre moderne)

« Quand le noyau a fini, il confie le système à **systemd**. »

### 📌 Rôle de systemd

- Démarre les services (réseau, SSH, affichage…)
- Gère les dépendances
- Remplace l'ancien système **init**

# Runlevels vs Targets (point central du cours)

⇒ **Contexte historique**

**Avant systemd, Linux utilisait SysVinit.**

👉 **Le système était contrôlé par des runlevels (0 à 6)**
👉 **Les commandes permettaient de :**

- **voir le runlevel courant**
- **changer de runlevel**
- **démarrer / arrêter des services**

---

## Commandes principales avec les runlevels (SysVinit)

---

🔷 **1. Voir le runlevel actuel**

✅ **Commande :**

```
runlevel
```

📌 **Sortie typique :**

```
N 5
```

🧠 **Interprétation :**

- **N : pas de runlevel précédent (au démarrage)**
- **5 : runlevel actuel (mode graphique)**

---

🔷 **Alternative :**

```
who -r
```

---

🔷 **2. Changer de runlevel**

✅ **Commande principale :**

```
init 3
```

**ou**

```
telinit 3
```

📌 **Exemples:**

| Commande | Effet |
|---|---|
| `init 0` | Arrêt |
| `init 1` | Mode secours |
| `init 3` | Mode texte |
| `init 5` | Mode graphique |
| `init 6` | Redémarrage |

> 💡 `telinit` est une interface vers `init`

---

🔷 **3. Démarrer / arrêter des services (avant systemd)**

**Les services étaient gérés via des scripts dans /etc/init.d/**

▶️ **Démarrer un service**

```
/etc/init.d/ssh start
```

⏹️ **Arrêter un service**

```
/etc/init.d/ssh stop
```

### 🔁 Redémarrer

```
/etc/init.d/ssh restart
```

---

---

### 🔷 4. Activer / désactiver un service selon le runlevel

**Sur Debian / Ubuntu :**

```
update-rc.d apache2 defaults
```

```
update-rc.d apache2 remove
```

---

**Sur Red Hat / CentOS :**

```
chkconfig apache2 on
```

```
chkconfig apache2 off
```

---

**Tableau comparatif : Avant / Après systemd**

| Action | SysVinit (avant) | systemd (aujourd'hui) |
|---|---|---|
| Voir mode actuel | `runlevel` | `systemctl get-default` |
| Changer de mode | `init 3` | `systemctl isolate multi-user.target` |
| Mode graphique | `init 5` | `graphical.target` |
| Arrêt | `init 0` | `systemctl poweroff` |
| Redémarrage | `init 6` | `systemctl reboot` |
| Gérer services | `/etc/init.d/` | `systemctl start/stop` |

---

**Compatibilité actuelle (point important)**

👉 **Sur Ubuntu moderne :**

```
init 3
```

**fonctionne encore ❗**
**Mais en réalité :**

- **init est un lien vers systemd**
- **systemd traduit 3 → multi-user.target**

---

🧠 **Phrase clé**

« *Avant systemd, on utilisait init et les runlevels. Aujourd'hui, on utilise systemctl et les targets, mais Linux garde la compatibilité*. »

---

🧪 **Mini-exercice**

```
ls -l /sbin/init
```

👉 **Question :**
**Pourquoi /sbin/init pointe vers systemd ?**

✍️ **Réponse attendue :**
**Pour assurer la compatibilité avec les anciens systèmes et scripts.**

---

🎓 **Conclusion**

✔ **Runlevels = ancienne méthode**
✔ **Targets = méthode moderne**
✔ **Les commandes SysVinit restent importantes à connaître pour la culture Linux**

### ⬅️ Les anciens Runlevels (SysVinit)

| Runlevel | Description |
|---|---|
| 0 | Arrêt |
| 1 | Mode secours |
| 3 | Mode texte |
| 5 | Mode graphique |
| 6 | Redémarrage |

### ➡️ Les Targets (systemd)

« systemd remplace les runlevels par des **targets**. »

| Target | Équivalent |
|---|---|
| `poweroff.target` | Runlevel 0 |
| `rescue.target` | Runlevel 1 |
| `multi-user.target` | Runlevel 3 |
| `graphical.target` | Runlevel 5 |
| `reboot.target` | Runlevel 6 |

### 📌 Message clé :

« Aujourd'hui, on parle de **targets**, pas de runlevels. »

# Commandes simples à montrer en TP

## 🔍 Voir le target actuel

```
systemctl get-default
```

### 🔁Changer le target par défaut

```
sudo systemctl set-default multi-user.target
```

### ▶️Passer temporairement en mode graphique

```
sudo systemctl isolate graphical.target
```

---

## Résumé

🧠 **Phrase magique à mémoriser :**

« Le BIOS démarre GRUB,
GRUB démarre le noyau,
le noyau lance systemd,
systemd démarre les services. »

---

# TP 1 – Comprendre le processus de démarrage Linux

## 🎯 Objectifs

- Identifier les étapes du démarrage Linux
- Reconnaître le rôle du BIOS/UEFI, GRUB, noyau et systemd

---

## 🔷 Partie A – Questions théoriques

### Exercice 1

**Question :**
Citer dans l'ordre les étapes du démarrage d'un système Linux.

---

### Exercice 2

**Question :**
Quel est le rôle du BIOS/UEFI ?

---

## Exercice 3

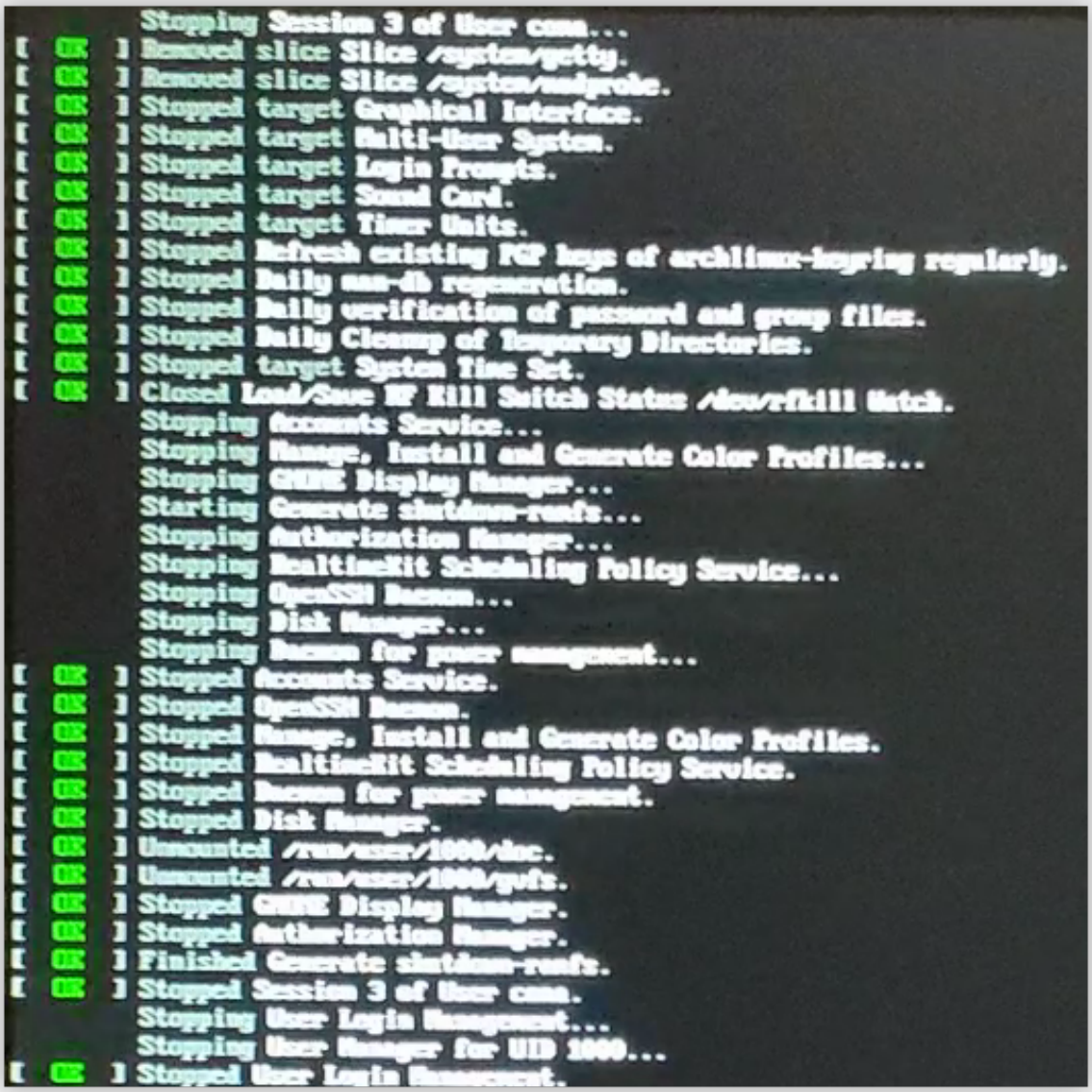
**Question :**
Quel programme est chargé par le BIOS/UEFI ?

---

# 🔷 Partie B – Observation pratique (VM Ubuntu)




System Time Synchronized
Load AppArmor profiles
Raise network interfaces
udev Kernel Device Manager
Show Plymouth Boot Screen
Show Plymouth Boot Screen
Forward Password Requests to Plymouth Directory Watch
Paths.
Local Encrypted Volumes.
Load/Save Screen Backlight Brightness of backlight:intel_backlight
Load/Save Screen Backlight Brightness of backlight:intel_backlight.
Reached target Swap
Reached target System Initialization.
Started Daily apt download activities.
Started Daily apt upgrade and clean activities.
Started Trigger anacron every hour.
Reached target Sockets.
Started Daily rotation of log files.
Started Daily man-db regeneration.
Reached target Basic System.
Starting Disk Manager...
Started Regular background program processing daemon.
Reached target Login Prompts.
Started D-Bus System Message Bus.
Starting Restore /etc/resolv.conf if the system crashed before the new link was cr
Starting LSB: Device...
Starting System Logging Service...
Starting Network Manager...
Starting Bluetooth service...
Started Daily Cleanup of Temporary Directories.
Reached target Timers.
Starting Accounts Service...




## Exercice 4

**Manipulation :**

1. Démarrer la VM
2. Observer le menu GRUB
3. Démarrer normalement

**Question :**
Que permet de choisir GRUB ?

---

# TP 2 – systemd et targets

## 🎯 Objectifs

- Comprendre systemd

- Identifier le target par défaut
- Passer d'un target à un autre

---

# 🔷 Partie A – Commandes de base

## Exercice 5

**Commande :**

```
ps -p 1
```

**Question :**
Quel est le rôle du processus PID 1 ?

---

## Exercice 6

**Commande :**

```
systemctl get-default
```

**Question :**
À quoi correspond le target affiché ?

---

# 🔷 Partie B – Targets (équivalent des runlevels)

## Exercice 7

Compléter le tableau suivant :

| Target | Équivalent |
|---|---|
| **poweroff.target** | ? |
| **rescue.target** | ? |
| **multi-user.target** | ? |
| **graphical.target** | ? |
| **reboot.target** | ? |

---

# TP 3 – Changer de mode de démarrage

## 🎯 Objectifs

- Changer le target par défaut
- Comprendre les conséquences

---

## 🔷 Partie A – Changement temporaire

**Exercice 8**

**Commande :**

```
sudo systemctl isolate multi-user.target
```

**Questions :**

1. Que devient l'interface graphique ?
2. Le changement est-il permanent ?

---

## 🔷 Partie B – Changement permanent

**Exercice 9**

**Commande :**

```
sudo systemctl set-default multi-user.target
```

**Question :**
Que se passe-t-il au prochain redémarrage ?

---

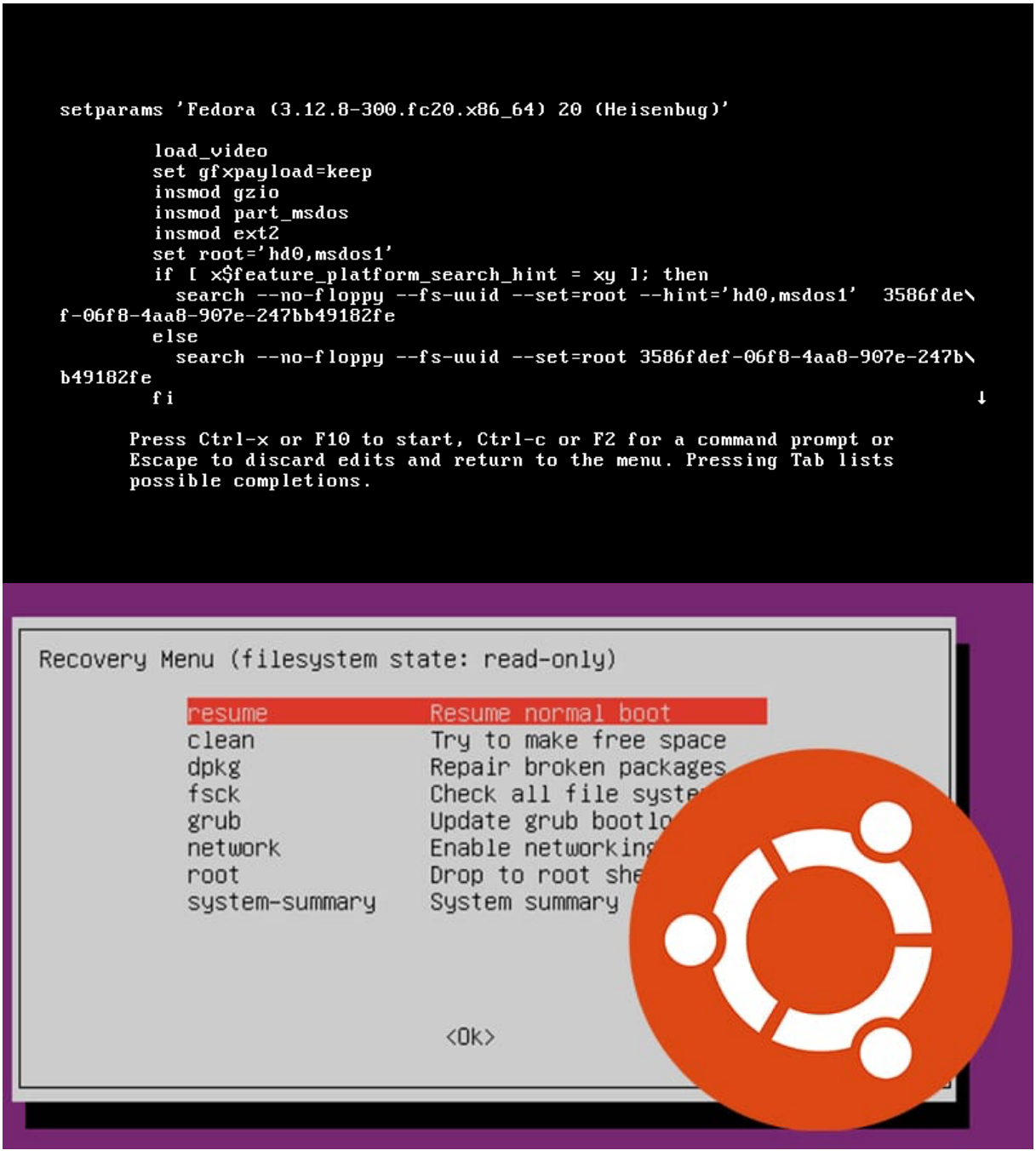
**Exercice 10**

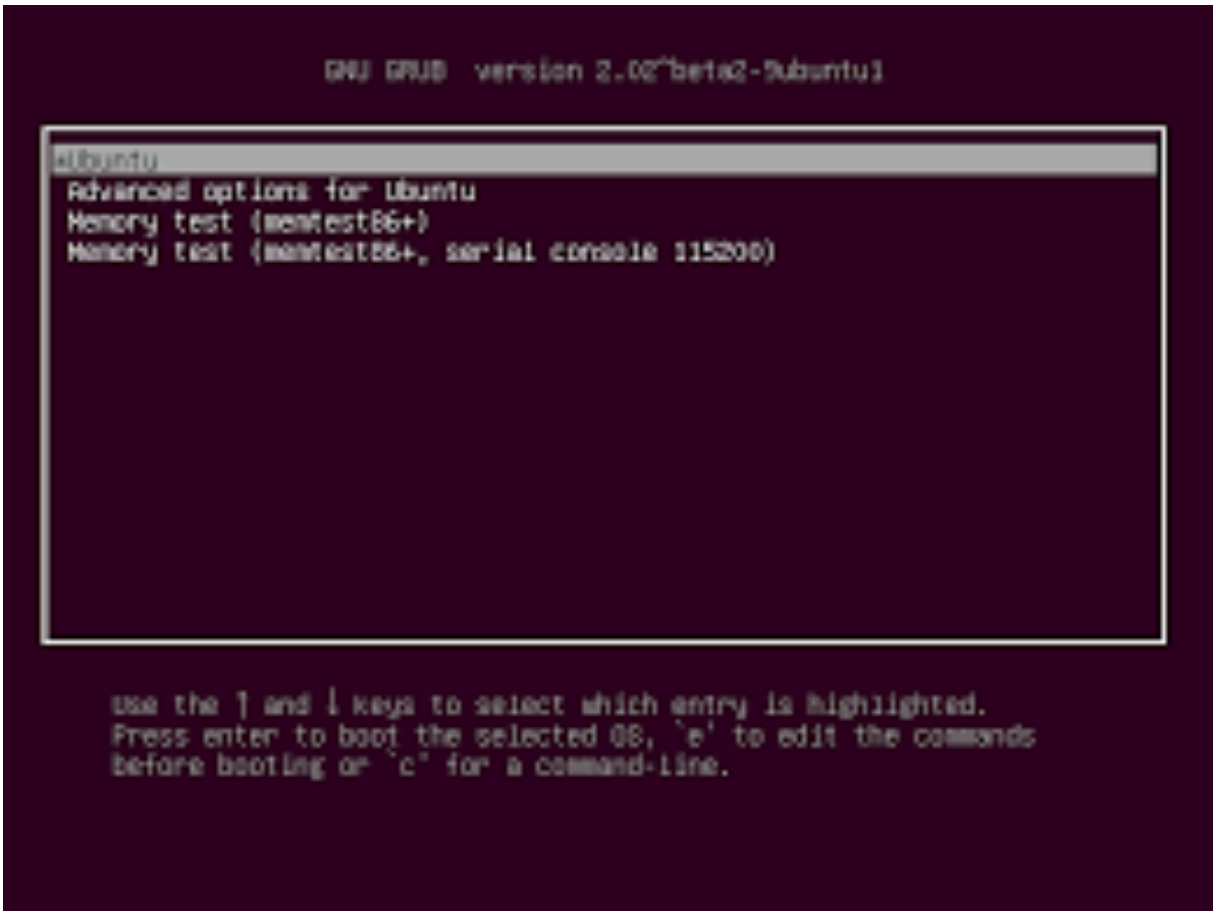
**Commande :**

```
sudo systemctl set-default graphical.target
```

---

# TP 4 – GRUB et démarrage (niveau découverte)






## Exercice 11

**Question :**
À quoi sert le mode recovery accessible via GRUB ?

---

## Exercice 12

**Question de réflexion :**
Pourquoi Linux permet-il de démarrer sans interface graphique ?

---

## 🆘 Qu'est-ce que `rescue.target` ?

👉 `rescue.target` est un **target de systemd** qui démarre le système avec **le strict minimum**.

💡 **Idée simple pour les étudiants :**
`rescue.target` = *le mode urgence pour réparer Linux quand ça ne démarre plus normalement*.

---

# 🔧 Que démarre `rescue.target` ?

### ✅ Démarré

- Le noyau Linux
- Le système de fichiers racine `/`
- Un **shell root**
- Quelques services essentiels (montage, console)

### ❌ Non démarré

- Interface graphique ❌
- Réseau (en général) ❌
- Services non essentiels (SSH, Apache, etc.) ❌
- Multi-utilisateurs ❌

### 👉 Un seul utilisateur : root

---

# 📊 Comparaison avec les autres targets

| Target | Description |
|---|---|
| `graphical.target` | Mode graphique |
| `multi-user.target` | Mode texte (serveur) |
| 👉 `rescue.target` | **Mode dépannage (1 utilisateur)** |
| `emergency.target` | Urgence extrême (encore plus minimal) |

---

## 🆚 `rescue.target` VS `emergency.target`

| rescue.target | emergency.target |
|---|---|
| **Système de fichiers monté** | Racine parfois non montée |
| **Services minimums** | Aucun service |
| **Utilisé pour réparations classiques** | Utilisé si le système est très endommagé |

---

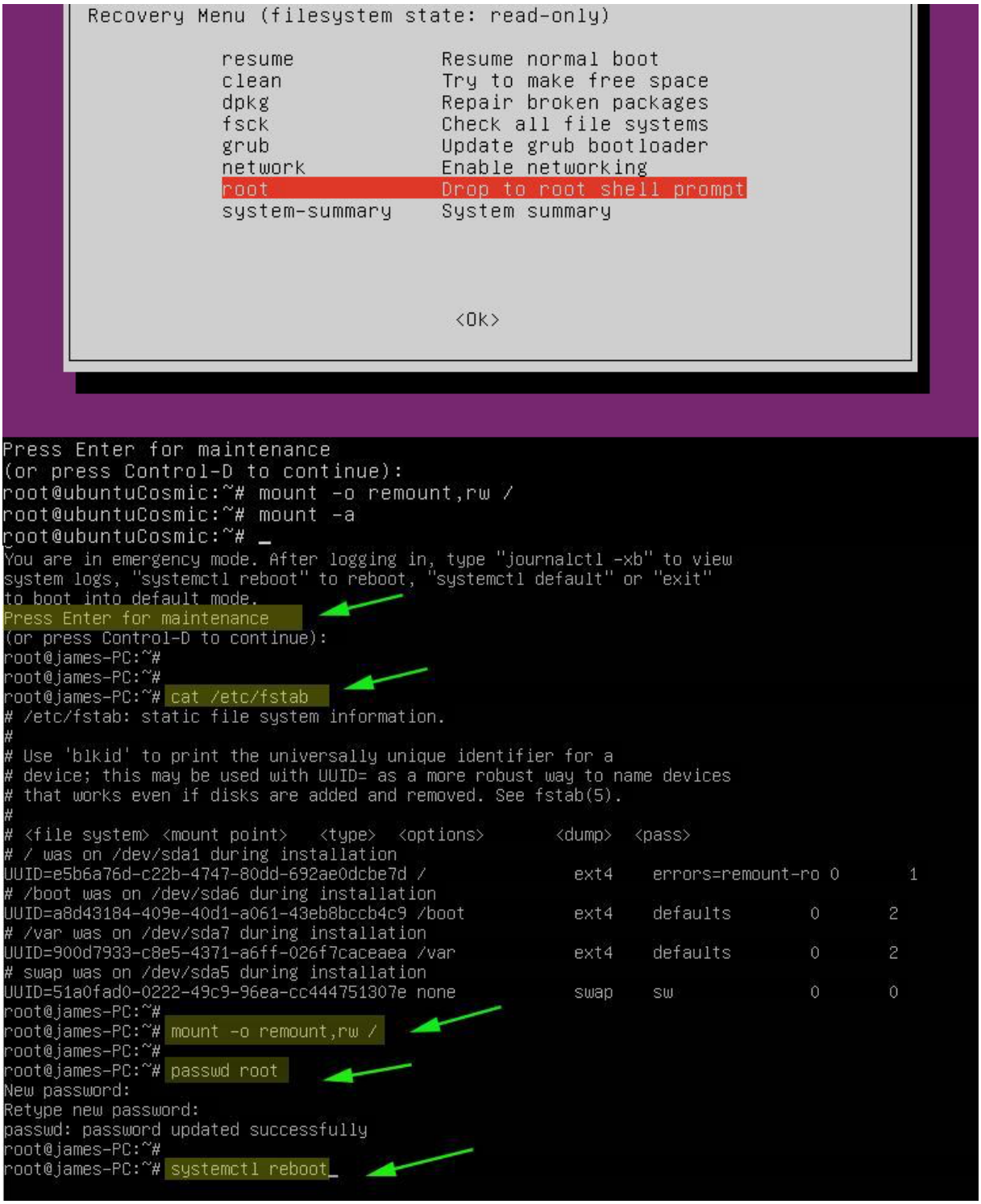
# 🖥️ Comment accéder à `rescue.target` ?

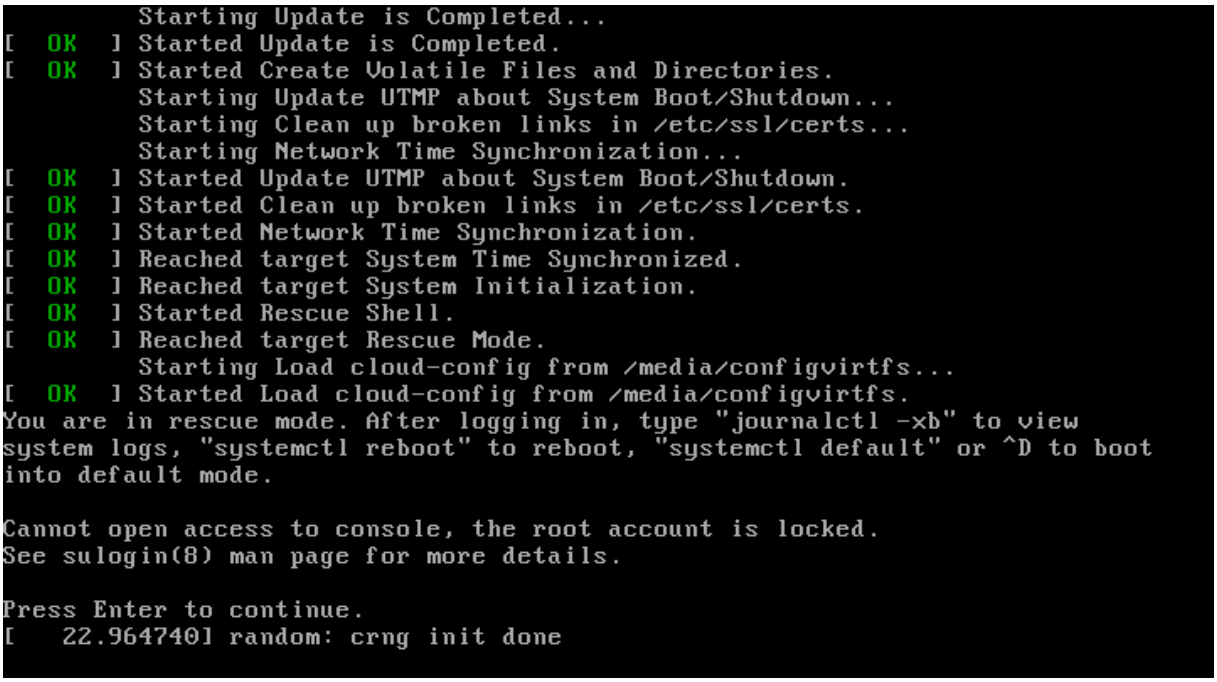
### 🔷 Depuis le système en fonctionnement

```
sudo systemctl isolate rescue.target
```

---

### 🔷 Depuis GRUB (quand Linux ne démarre plus)


Recovery Menu (filesystem state: read-only)
resume Resume normal boot
clean Try to make free space
dpkg Repair broken packages
fsck Check all file systems
grub Update grub bootloader
network Enable networking
root Drop to root shell prompt
system-summary System summary
<Ok>
Press Enter for maintenance
(or press Control-D to continue):
root@ubuntuCosmic:~# mount -o remount,rw /
root@ubuntuCosmic:~# mount -a
root@ubuntuCosmic:~#
You are in emergency mode. After logging in, type "journalctl -xb" to view
system logs, "systemctl reboot" to reboot, "systemctl default" or "exit"
to boot into default mode.
Press Enter for maintenance
(or press Control-D to continue):
root@james-PC:~#
root@james-PC:~#
root@james-PC:~# cat /etc/fstab
# /etc/fstab: static file system information.
#
# Use 'blkid' to print the universally unique identifier for a
# device; this may be used with UUID= as a more robust way to name devices
# that works even if disks are added and removed. See fstab(5).
#
# <file system> <mount point> <type> <options> <dump> <pass>
# / was on /dev/sda1 during installation
UUID=e5b6a76d-c22b-4747-80dd-692ae0dcbe7d / ext4 errors=remount-ro 0 1
# /boot was on /dev/sda6 during installation
UUID=a8d43184-409e-40d1-a061-43eb8bccb4c9 /boot ext4 defaults 0 2
# /var was on /dev/sda7 during installation
UUID=900d7933-c8e5-4371-a6ff-026f7caceaea /var ext4 defaults 0 2
# swap was on /dev/sda5 during installation
UUID=51a0fad0-0222-49c9-96ea-cc444751307e none swap sw 0 0
root@james-PC:~#
root@james-PC:~# mount -o remount,rw /
root@james-PC:~#
root@james-PC:~# passwd root
New password:
Retype new password:
passwd: password updated successfully
root@james-PC:~#
root@james-PC:~# systemctl reboot




1. Redémarrer
2. Dans GRUB → **Advanced options**
3. Choisir **Recovery mode**
4. Sélectionner **root – Drop to root shell**

➡️ Le système démarre en **rescue.target**

---

# 🛠️ À quoi sert concrètement `rescue.target` ?

✔ Réinitialiser un mot de passe root
✔ Réparer `/etc/fstab`
✔ Supprimer un service bloquant le démarrage
✔ Vérifier le disque (`fsck`)
✔ Dépanner un système qui boucle au démarrage

# Cas pratique de changement de Mot de passe utilisateur en mode rescue.target

```
You are in rescue mode. After logging in, type "journalctl -xb" to view
system logs, "systemctl reboot" to reboot, "systemctl default" or "exit"
to boot into default mode.
Donnez le mot de passe du superutilisateur pour la maintenance
(ou appuyez sur Ctrl et D pour continuer) :
Identifiant incorrect

Donnez le mot de passe du superutilisateur pour la maintenance
(ou appuyez sur Ctrl et D pour continuer) :
root@test-VirtualBox:~# passwd
Nouveau mot de passe :
MOT DE PASSE INCORRECT : Le mot de passe est un palindrome
Retapez le nouveau mot de passe :
Les mots de passe ne correspondent pas.
Nouveau mot de passe :
MOT DE PASSE INCORRECT : Le mot de passe comporte moins de 8 caractères
Retapez le nouveau mot de passe :
passwd : mot de passe mis à jour avec succès
root@test-VirtualBox:~# passwd test
Nouveau mot de passe : _
```

```
You are in rescue mode
root@test-VirtualBox:~#
```

👉 Mode secours (rescue.target)

👉 Vous avez **un shell root**

👉 La commande `passwd` fonctionne **sans mot de passe préalable**

## Principe clé à comprendre (TRÈS IMPORTANT)

### 🧠 Linux part du principe suivant :

**Si quelqu'un a un accès physique ou console à la machine, elle n'est plus sécurisée.**

👉 La sécurité Linux **ne protège PAS contre quelqu'un qui contrôle physiquement la machine**, sauf configuration spéciale.

---

**Pourquoi c'est NORMAL (et voulu)**

**Le mode rescue est un mode de récupération**

Il sert à :

- réparer un système cassé
- récupérer l'accès quand **on a perdu le mot de passe**
- réparer /etc/fstab, le disque, GRUB, etc.

**Si le mode rescue demandait un mot de passe… il ne servirait à rien**

---

**Accès au rescue = accès root**

Entrer en rescue mode signifie :

- accès direct à la console
- système monté (parfois en lecture seule)

- **droits root**

Donc **passwd est autorisé**, c'est normal.

---

**La vraie question de sécurité est ailleurs**
**Faux problème**
« Linux n'est pas sécurisé »
**Vrai problème**
« Qui peut accéder au mode rescue ? »

---

**Où se trouve la VRAIE sécurité**
**Mot de passe GRUB (TRÈS IMPORTANT)**
Sans mot de passe GRUB :

- n'importe qui peut :
  - appuyer sur e
  - modifier la ligne du noyau
  - ajouter init=/bin/bash
  - devenir root

**C'est exactement ce qui se passe ici**
**Solution : protéger GRUB**

---

**Chiffrement du disque**
Si le disque est chiffré :

- même en rescue mode
- **les données sont illisibles sans la clé**

Même root ne peut rien faire sans le mot de passe de chiffrement

---

**Sécurité physique**

- accès BIOS/UEFI protégé par mot de passe
- boot USB/CD désactivé
- accès serveur contrôlé

---

**Cas particulier : VirtualBox (très important)**
**Vous êtes dans une machine virtuelle**
Dans ce contexte :

- l'admin de l'hôte **est Dieu**
- la VM n'est **pas censée être sécurisée contre son propriétaire**
- c'est NORMAL en TP, BTS, labo, PFE

---

**Résumé clair pour vos étudiants**
Vous pouvez leur dire :
**Linux est sécurisé tant que l'accès physique est contrôlé.**
**Si quelqu'un a accès à la console ou au boot, la sécurité logicielle seule ne suffit plus.**
La vraie sécurité repose sur :

- mot de passe GRUB
- chiffrement du disque
- sécurité physique

---

**Phrase parfaite pour un examen / cours**
*Le mode rescue permet volontairement l'accès root afin de réparer le système.*
*La sécurité ne repose pas sur l'interdiction du rescue mode, mais sur le contrôle de l'accès au démarrage et au matériel.*

# 🧠 Phrase clé

**« `rescue.target`, c'est Linux en mode dépannage :**
**un seul utilisateur, pas de graphique, juste l'essentiel pour réparer. »**

# 🧠 Est ce qu'il n'y a pas de runlevel 2 et 4

La réponse courte est : **oui et non**, selon **le système d'init utilisé**.

---

🔢 **Les runlevels : 0 à 6 (théorie)**
Dans l'ancien système **SysVinit**, il existe **7 runlevels** :

| Runlevel | Signification |
|---|---|
| **0** | Arrêt |
| **1** | Mode mono-utilisateur (secours) |
| **2** | Multi-utilisateur (variable) |
| **3** | Multi-utilisateur (texte) |
| **4** | **Non défini / libre** |
| **5** | Multi-utilisateur + graphique |
| **6** | Redémarrage |

👉 **Donc oui, les runlevels 2 et 4 existent officiellement.**

---

❓ **Alors pourquoi on dit souvent qu'ils "n'existent pas" ?**
Parce que **leur usage dépend de la distribution Linux**.

---

🐧 **Cas de Debian / Ubuntu (le plus courant en BTS)**
Sur **Debian et Ubuntu** :

| Runlevel | Réalité |
|---|---|
| **2** | ✔ Existe, **équivalent au 3** |
| **3** | ✔ Existe |
| **4** | ✔ Existe mais **non utilisé** |
| **5** | ✔ Existe |

👉 En pratique :

- **2, 3 et 4 font exactement la même chose**
- Seuls **1, 5 et 0/6** ont un vrai rôle distinct

💡 **Message simple :**
« Sur Ubuntu, runlevel 2 = 3 = 4 »

### 🦅 Cas de Red Hat / CentOS / Rocky

Sur ces distributions :

| Runlevel | Usage |
|---|---|
| **2** | Peu utilisé |
| **3** | Mode texte |
| **4** | Libre (admin) |
| **5** | Mode graphique |

👉 Ici, **3 et 5 sont vraiment différents**

---

### 🔁 Et avec systemd aujourd'hui ?

Avec **systemd**, les runlevels sont **obsolètes** mais **encore supportés pour compatibilité**.

**Équivalence officielle systemd**

| Runlevel | Target systemd |
|---|---|
| **0** | poweroff.target |
| **1** | rescue.target |
| **2** | multi-user.target |
| **3** | multi-user.target |
| **4** | multi-user.target |
| **5** | graphical.target |
| **6** | reboot.target |

👉 Voilà pourquoi **2 et 4 semblent "inutiles"** :
ils pointent vers **le même target**.

---

### 🧠 Phrase clé à retenir (examen)

**« Les runlevels 2 et 4 existent, mais sur Ubuntu ils n'ont pas de rôle spécifique et sont équivalents au runlevel 3.**
**Avec systemd, ils sont remplacés par des targets. »**

---

### 🧪 Petit exercice

runlevel
systemctl get-default
👉 Question :
« Pourquoi la commande runlevel affiche N 5 même si on utilise systemd ? »
✍️ **Réponse attendue :**
Parce que systemd assure la compatibilité avec l'ancien modèle SysVinit.

---

### 🎓 Conclusion

✔ Oui, **runlevel 2 et 4 existent**
✔ Mais **souvent inutilisés**